

A checker for schedule-dependent work amplification in Hydro programs

The problem

A distributed program does work in response to its input, and in a well-behaved program the amount of work is a function of that input alone. Many real programs do not have this property. A client that re-sends a request after a timeout, a cache that fetches from its origin whenever a lookup misses, a follower that starts an election when a heartbeat is late, a peer that repeats a gossip message until it is acknowledged: each of these reacts to the lateness or absence of a message by generating more messages or more work. Under a favourable delivery schedule the reaction never fires and the program does exactly the work its input requires. Under an unfavourable one the same input costs more, and the extra work can itself delay the messages whose lateness caused it. When that feedback is strong enough, a transient delay pushes the system into a state where it spends its whole capacity on redundant work, completes nothing, and does not recover after the delay is gone. We call the general property schedule-dependent work amplification and the self-sustaining case metastability.

This property is hard to test for. The schedules that expose it are rare under random exploration, because they require the same edge to be delayed for many consecutive steps while everything else proceeds normally, and the probability of that under independent random choices falls geometrically with the length of the delay. The property also depends on timing in a way that fixed tests cannot probe: a load test fixes a delivery schedule and varies the load, a unit test fixes both, and neither asks whether some other schedule would have made the same load cost more. What is needed is a way to search over schedules while holding the input fixed, and a way to measure work that does not depend on knowing what the program is for.

The idea, and why it works

The checker rests on five claims, each argued below and each demonstrated on one program. The program is a request-response service with client-side retry. The client forwards each application request to the server with an id, keeps a table of requests awaiting a response, re-sends any request unanswered within 40 of its clock ticks, and gives up after three sends. The server keeps arriving requests in a first-in, first-out queue and serves at most five per tick. The workload throughout is two requests per round against that capacity of five, so under an undisturbed schedule there is no backlog and no re-send ever fires.

The simulator's decisions are an adversarial network's moves

Hydro is a dataflow language for distributed programs whose compiler also produces a deterministic simulator. A Hydro program cannot read a wall clock, so every timer is an input stream of tick events, and in simulation the harness, not the operating system, decides when each tick event arrives. Each process advances in discrete atomic steps (Hydro calls one such step a tick, and the block of dataflow that runs inside it a `sliced!` block), and a step runs only when something is waiting for it. At the boundary of each step the simulator makes two kinds of decision. Wherever messages wait to enter a step, it decides which of the waiting messages are delivered into this step and which stay waiting (this point is a `use::batch` call in the program, and we call it a batch decision point). Wherever a step reads a piece of state that is updated by another step, it decides how fresh a version of that state the step sees (this point is a `use::snapshot` call, and we call it a snapshot decision point). Every network message that takes part in a process's logic passes through a batch decision point at its receiver, and there is no other place in the simulation where nondeterminism enters.

These two decisions are exactly what an adversarial network controls and nothing more. A network can delay a message or a batch of messages by any amount and then deliver it, and a network can cause a process to act on a stale view of another process's state by delaying the update that would have refreshed it. It cannot invent messages, alter their contents, or change what a process does with what it receives. The simulator's decisions have the same reach: a batch decision point can only delay, a snapshot decision point can only stale, and the program's logic runs unchanged on whatever it is given. A schedule of these decisions is therefore a complete description of one adversarial network behaviour, and searching over schedules is searching over

adversaries. In the retry service the relevant decision points are the server's admission of requests, the client's admission of responses, the client's admission of its own clock ticks, and a few bookkeeping points that report metrics; holding the client's response point for a while is what a network does when it delays the server's replies.

Delaying one point at a time isolates its effect

The checker fixes every decision except one. It first runs the program once under a deterministic reference schedule in which every batch decision point admits everything that is waiting, every snapshot decision point sees the newest version, and steps run in a fixed round-robin order (we call this the prompt schedule). That run establishes the baseline counts and discovers every decision point that ever has something waiting. Then, for each discovered decision point and each length k in a grid, it runs the program again on the same input with that one point held for k consecutive rounds starting from a fixed round, while every other decision still follows the prompt schedule. A held batch point delivers nothing during the hold and everything at once when it ends, which is what a delayed network edge looks like from inside the program. A held snapshot point keeps showing its step the version it saw when the hold began and jumps to the newest version when the hold ends, which is what a delayed state update looks like.

Because the simulator is deterministic and the input is fixed, the held run and the baseline run are identical up to the round at which the hold begins, and every difference between them after that round is caused by the hold. Because only one point is held and the reference schedule elsewhere is the most prompt one possible, nothing in the held run is delayed except what the held point delays and what the program does in consequence. This is what makes the comparison an experiment rather than an observation: the difference between the two runs is the effect of one delay on one edge, with no other perturbation to confound it.

Making a hold of any length a single decision is also what makes the search reach the schedules that matter. A uniform random scheduler decides independently at each step whether a waiting item is released, so the chance that one item survives forty consecutive decisions is on the order of one in a trillion, and a program whose timeout is forty ticks is out of reach of that search entirely. Holding a named point for a chosen number of rounds reaches a forty-round delay as cheaply as a five-round one.

Records admitted and messages sent cannot rise under a delay unless the program derived them

Work is measured passively inside the simulator, with no instrumentation of the program and no knowledge of what its messages mean. Two counts are read over the whole run. The first is the number of records admitted through batch decision points, summed across the program. The second is the number of network messages each process or cluster member sends, kept separately per sender.

Both counts have the property that a delay alone cannot raise them. Over a whole run, a batch point's admitted total is the number of records that arrived at it minus the number still waiting at the end. A hold changes when those records are admitted, in fewer and larger groups, and it may push some past the end of the run, which lowers the total; it does not change how many arrived. The only way the total rises is that more records arrived, and records arrive only because some step upstream produced them. The same holds for messages: a message is sent when the sender's dataflow emits it, and delaying the sender's inputs cannot make its logic emit more unless the logic itself reacts to the delay. A positive difference between the held run and the baseline run on the same input is therefore, by construction, work that the program derived in response to being delayed, which is what redundant work means here. The measure needs no notion of a request, a completion, or a retry; it needs only the fact that delay conserves records.

The retry service shows the count doing exactly this. Holding the client's response point for 0, 5, 10, 20, 40, 60, 80 and 100 rounds produces, in extra network messages against the baseline, the curve 0 0 0 0 0 80 160 320. Every number in it can be derived by hand. A reply delayed by less than the 40-tick timeout needs no re-send, so holds of 5 through 40 add nothing. Beyond 40, each of the two requests per round issued during the excess is re-sent once and served once more, which is four extra messages per round: a hold of 60 exceeds the timeout by 20 rounds and costs 4 times 20, which is 80, and a hold of 80 costs 160. At 100, the requests issued in the first 20 rounds of the hold have waited more than 80 ticks and are re-sent a second

time, adding another 80 for 320. The checker's count and the hand derivation agree to the message, and the count was produced without the checker knowing that the program has requests, replies, or a timeout. Read in admitted records rather than messages the same curve is 0 0 0 0 200 400 800, because each re-sent request and its reply pass through five decision points in all, and the report shows both.

The point whose delay produces the most extra work is where the reaction lives

The verdict is hazardous if any hold of any discovered decision point raises either count above the baseline, and benign otherwise. Messages are read per sender because a cluster total can fall while one member's traffic rises, as when a leader stops sending heartbeats while its followers start sending vote requests. Admitted records are read in total because within one process a reaction can replace one record with another, as when a completion becomes an abandonment, without adding work. The decision point whose hold produces the largest gain is reported as the location of the hazard.

That attribution is sound for the following reason. Extra work appears in a held run only if the program's logic, somewhere, turns the lateness of the held records into new records. The held point is the only place lateness was introduced, so the reaction is to the records that pass through that point, and its size measures how strongly the program reacts to delay on that particular edge. Holding a point the program does not react to produces zero at every length, however long the hold, because the records are the same and merely later. Comparing gains across points therefore ranks the program's edges by how much lateness on each one costs, and the largest is the edge the program is most sensitive to. The report also lists which other decision points admitted more records under the winning hold, which names the path the extra records took.

In the retry service, holding the server's request arrivals point produces a larger gain than holding the client's responses: 588 extra messages at a hold of 100 against 320. The mechanism explains the difference. Requests held at the server time out at the client, their re-sends queue behind the held originals, and when the hold ends the server receives all of them at once, so its queueing delay stays above the client's timeout for many rounds after the hold has ended and the re-send storm feeds itself for a while before draining. The checker names this point, at the line of the server's `use:batch` on its request stream, as the location, and lists the client's response point and the metrics points as the places the extra records crossed. The clock point and the metrics points, held for any length, produce zero, because the program does nothing in response to their lateness except wait. The same program with the retry limit set to one send, so that a timed-out request is abandoned rather than re-sent, produces zero at every hold length on every point, and the verdict is benign: the only edge whose lateness the program reacted to has had its reaction removed.

A gain that grows with the delay is the signature of feedback

The curve over hold lengths carries more than the verdict. A program that pays a fixed cost for a delay of any length, say one re-send per delayed request however late the reply, produces a step: zero below the threshold and a constant above it. A program whose reaction to delay produces further delay produces a curve that keeps rising with the hold, because each round of excess lateness generates work that itself lengthens the lateness of what follows. The response-point curve of the retry service rises linearly past the timeout and then doubles its slope at twice the timeout, as second re-sends begin, which is the arithmetic of a fixed rule applied to a growing set of late requests. The arrivals-point curve rises faster than that arithmetic predicts, because the re-sends it provokes lengthen the server's queue and thereby delay the requests behind them, which is the feedback loop that makes retry storms self-sustaining under load. Two shapes appear elsewhere in the corpus described below: a plateau, where the program caps its own reaction at a fixed rate and the storm continues at that rate for the rest of the run, and a rise followed by a fall proportional to the rounds remaining, which is what a program looks like once a hold has tipped it into a state it does not leave. The grid of hold lengths exists so that the report shows this shape and not only its sign.

What the user provides and what comes back

The user provides three things. The first is the program, written in ordinary Hydro dataflow with its timers as stream parameters, which is how any Hydro program must be written to be simulated at all; a deployment wires periodic timers into the same parameters. The second is a closure that sends one round of a steady

workload into the program's inputs, including one tick event to each timer; the checker calls it once per round and then runs the simulation until nothing more can happen before calling it again. The workload should be the program's ordinary operating point with no burst, because the question the checker answers is whether some schedule makes that same input cost more. The third is a configuration: the number of rounds, the round at which holds begin, and the grid of hold lengths, with defaults of 240 rounds, holds from round 20, and the grid 0, 5, 10, 20, 40, 60, 80 and 100.

What comes back is a report. It contains the verdict, the baseline counts, and one curve per discovered decision point giving the extra admitted records and the extra messages from each sender at every hold length. For a hazardous program it also contains a location: the decision point whose hold produced the largest gain, given as a source file and line, together with what rose (messages from which sender, or admitted records), by how much, at which hold length the rise first appeared, and which other decision points admitted more records under that hold. Because every decision point in one step reports the location of the step's block, the location is a block and an index within it rather than a single line. The simulation is compiled once and reused for every run, after which a 240-round run takes well under a second; the twenty-two corpus configurations described next, each with between two and nine decision points and eight hold lengths, are checked together in 63 seconds of wall clock.

Evaluation

The checker was evaluated against a corpus of twelve small Hydro programs written for the purpose by authors who were given a specification of the property and instructed to know nothing about how it would be detected. The programs cover request-response retry with and without exponential backoff, the same service with a bounded and an unbounded server queue, a read-through cache with a time-to-live with and without coalescing of concurrent misses, delta gossip with and without acknowledgement and re-send, full-state gossip of a grow-only set, heartbeat-timeout leader election, lease renewal against a single server, work-queue rebalancing on stale length reports, a log store whose compaction runs on leftover budget, a pure heartbeat, and a semi-naïve transitive closure. Most programs carry a parameter that changes the mechanism under study, which gives twenty-two configurations in all.

Each configuration carries a label with a stated basis. A configuration is labeled hazardous by exhibited collapse when a test drives it, on the fixed deterministic schedule, through a bounded burst of extra load and shows in the tail of the run, long after the burst has ended, that redundant work persists: a backlog that keeps growing, completions at zero, or a storm of messages still running. For the retry service a sixty-round burst leaves the server backlog growing from 1038 to 1237 over the last two hundred rounds with zero completions against a baseline of four hundred. A configuration is labeled benign by assurance argument when its author wrote down why total work is a closed-form function of the input under every schedule and checked that closed form under a few hundred random schedules. Ten configurations are hazardous by exhibited collapse and seven are benign by assurance argument. Five more have neither basis: they carry the mechanism of a collapsing configuration together with a tuning parameter such as backoff, a queue bound, or a cooldown that kept the test's burst from collapsing them, and since a tuning parameter can only move the threshold at which a mechanism runs away, the checker is expected to find amplification in them without a label asserting that it must. One program, the log store, is excluded from the scorecard, because its author modeled the cost of a read as an integer inside a record rather than as records moved, so no work actually increases in it and there is nothing for a record count to see.

On the twenty scored configurations the checker agrees with all sixteen ground-truth labels and all four expectations, reading nothing but the simulator's own counts. In every hazardous case the decision point it names is the one the program's mechanism points at. For the three retry-based services it names the server's request arrivals first and the client's responses second. For the cache it names the point where fills arrive from the origin. For delta gossip and for lease renewal it names the point where acknowledgements arrive. For the election it names the point where messages enter a follower's step (in one of the two election configurations it names the follower's timer, one line away in the same block, which delays the same heartbeats), and the count that rose is one follower's outgoing messages, by 840, while the leader's traffic fell. For rebalancing it names the point where tasks arrive at a worker.

The curve shapes described above all occur. Delta gossip with re-send until acknowledged produced 0 100 7063 7065 7065 7065 7065 7065 extra re-sends and repeated merges under a hold on its acknowledgement point, a jump and then a plateau, because the program caps re-sends at one per peer per tick. The read-through cache without coalescing produced 336 388 468 2520 2668 2668 2668 2668 total fetches against a baseline of 336 under a hold on the point where fills arrive, with the jump where a held fill's age crosses the twenty-tick time-to-live and every fill arrives stale. The benign programs read flat or falling on every decision point; the transitive closure's candidate count under a hold on its step timer was 288 288 282 270 246 222 198 174, falling only because iterations were postponed past the end of the run.

This is agreement on the corpus against which the verdict rule was developed. The choice to read messages per sender and admitted records in total was made after a plain total missed the staggered election and a per-point breakdown wrongly flagged the one-attempt retry service, so the twenty agreements measure how well the rule fits the programs it was fitted to, and not how it will fare on programs it has not seen.

What the checker cannot do

The verdict is a statement about the workload the user supplied, and it can depend on that workload; the log store with a compaction reserve is not amplified by any schedule of one write per four reads and is amplified under three writes per two reads, so a program with a tuning parameter can pass at one operating point and fail at another, and the checker reports only the point it was given. Work that a program performs as arithmetic inside an operator, rather than as records that move, is invisible, because the counts see records entering steps and messages crossing the network and nothing that happens to a number inside a record or to records flowing between operators within one step. Decision points are held one at a time, from one fixed starting round, for the lengths in the grid, so a hazard that requires two edges to be delayed together, or a delay that must begin at a particular moment, is outside the search. The reported location is the step whose decision point was held, given as the source location of its block and the index of the point within it, rather than a single operator. A program with no batch or snapshot decision point offers the simulator nothing to hold and is reported benign on that basis alone, as the pure heartbeat is. Snapshot decision points are held but their reads are not counted as work, because a step reads its snapshot exactly once every time it runs regardless of freshness, and no program in the corpus makes its sends depend on a snapshot, so the snapshot hold has not yet met a case that could exercise it.

Where the code is

The checker is the function `hydro_lang::sim::amplification::check`, which takes a simulation, a `CheckConfig`, and a per-round workload closure and returns a `Report`. The schedule that holds one decision point lives in `hydro_lang::sim::hold_one_hook`, the passive counts in `hydro_lang::sim::work_counts`, and the reference schedule in `hydro_lang::sim::prompt_schedule`. The corpus programs and their labeling tests are under `hydro_test/src/cluster/witnesses/`, with the retry service one directory up in `hydro_test/src/cluster/rpc_retry.rs`, and the tests that run the checker against every corpus configuration and assert the recorded verdict and location are in `hydro_test/src/cluster/witnesses/checker_experiments.rs`. An independent evaluation on programs written by an author with no knowledge of the checker is under way.